RAMDEOBABA UNIVERSITY, NAGPUR

An Autonomous University Accredited with 'A+' Grade by NAAC

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (ARTIFICIAL INTELLIGENCE & DATA SCIENCE)

RBU
ESTD 1984

A PROJECT REPORT ON

HOUSEHOLD ELECTRICITY PATTERN ANALYSIS & PREDICTIVE MODELING

Distributed Big Data Pipeline with Hadoop MapReduce, Apache Spark, HDFS, Cassandra & Machine Learning

Submitted in partial fulfillment of the requirements for the laboratory course of
BIG DATA ANALYTICS LAB (V-SEMESTER, B.TECH CSE AI & DS)

SUBMITTED BY:

BHOMICK DEBIYA
Roll Number: **32**
B.Tech CSE (AI & Data Science)

SHOURYA CHILATE
Roll Number: **58**
B.Tech CSE (AI & Data Science)

UNDER THE GUIDANCE OF:

PROF. SONAM CHOPADE
Assistant Professor
Dept. of CSE (AI & Data Science)

PROF. SAMIHA KHAN
Assistant Professor
Dept. of CSE (AI & Data Science)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AI & DS)
ACADEMIC SESSION: 2026

CERTIFICATE OF APPROVAL

This is to certify that the project report entitled **"Household Electricity Pattern Analysis & Predictive Modeling"** submitted by **Bhomick Debiya (Roll No: 32)** and **Shourya Chilate (Roll No: 58)** to **Ramdeobaba University, Nagpur**, is a bona fide record of authentic academic work carried out by them in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering (Artificial Intelligence & Data Science) for the course of **Big Data Analytics Lab (V-Semester)** during the academic year 2026.

The project demonstrates end-to-end distributed Big Data engineering comprising multi-gigabyte ingestion into the Hadoop Distributed File System (HDFS), native MapReduce batch aggregation, Apache Spark in-memory transformations, dual-engine mathematical convergence verification, Apache Cassandra NoSQL storage modeling with graceful fault fallback, and machine learning load forecasting.

_____

Prof. Sonam Chopade
Project Guide / Faculty Mentor
Dept. of CSE (AI & DS)

_____

Prof. Samiha Khan
Project Guide / Faculty Mentor
Dept. of CSE (AI & DS)

_____

Head of Department
Dept. of CSE
Ramdeobaba University, Nagpur

_____

External Examiner
Big Data Analytics Lab Examination
Ramdeobaba University

CANDIDATE'S DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity.

Bhomick Debiya (Roll No: 32)

B.Tech CSE (AI & DS), Sem-V

Shourya Chilate (Roll No: 58)

B.Tech CSE (AI & DS), Sem-V

EXECUTIVE ABSTRACT

Modern smart electric grids generate continuous, high-velocity streams of time-series consumption readings at minute-level granularity. Traditional single-node relational databases (RDBMS) fail under multi-million row workloads due to I/O bottlenecks and memory saturation. This report presents an enterprise distributed Big Data analytics and predictive forecasting framework built over the **UCI Individual Household Electric Power Consumption dataset**, comprising **2,075,260 raw readings (132.9 MB)** collected over 47 consecutive months.

The engineered architecture deploys **Hadoop Distributed File System (HDFS)** for scalable distributed raw storage; native **Java Hadoop MapReduce** for disk-based 24-hour baseline aggregation; and **Apache Spark (PySpark 3.5.5)** for in-memory temporal feature engineering and sub-metering disaggregation. Crucially, a **Dual-Engine Cross-Validation** mechanism verifies that MapReduce and Spark produce mathematically identical hourly aggregates with a maximum discrepancy of **0.000000 kW** across all 24 hours. Furthermore, an **Apache Cassandra NoSQL** keyspace is modeled for low-latency serving with an automatic zero-crash fallback layer. A **Random Forest Regressor** forecasts active power demand with an RMSE of **0.8793 kW** (16.83% improvement over the baseline), integrated into an interactive **Gradio 6.0** web dashboard.

TABLE OF CONTENTS

1. Introduction & Industrial Context	**Page 4**
1.1 Motivation & Big Data Dimensions (The 4 V's)	**Page 4**
1.2 Core Project Objectives	**Page 4**
2. Dataset Profiling & 5-Stage Cleaning Pipeline	**Page 4**
2.1 Sensor Attributes & Physical Specifications	**Page 4**
2.2 Data Quality Funnel & Null Handling Strategy	**Page 5**
3. Distributed System Architecture & Implementation	**Page 5**
3.1 End-to-End Pipeline & Tiered Layout	**Page 5**
3.2 HDFS Distributed Storage Tier (Port 9000)	**Page 5**
3.3 Batch Processing: Java & Python MapReduce	**Page 6**
3.4 In-Memory Processing: Apache Spark (PySpark)	**Page 6**
3.5 Dual-Engine Mathematical Convergence Verification	**Page 7**
3.6 NoSQL Distributed Serving: Apache Cassandra	**Page 7**
3.7 Machine Learning Forecasting Engine (Random Forest)	**Page 8**
3.8 Gradio 6.0 Interactive Web Application	**Page 8**
4. Analytical Findings, Load Profiles & Physics Validation	**Page 9**
4.1 Diurnal 24-Hour Active Power Load Dynamics	**Page 9**
4.2 Weekday versus Weekend Temporal Behavioral Shift	**Page 9**
4.3 Appliance Sub-Metering Disaggregation Breakdown	**Page 10**
4.4 Multi-Sensor Correlation Heatmap & Electrical Physics	**Page 10**
5. Viva Voce Defense Guide, Conclusion & Future Scope	**Page 11**
6. References & Technology Standards	**Page 12**

1. INTRODUCTION & INDUSTRIAL CONTEXT

Electric power utilities are undergoing rapid digitalization through Advanced Metering Infrastructure (AMI). Modern smart meters record active power, reactive power, line voltage, current intensity, and appliance-specific sub-metering at high temporal frequency (e.g., minute-by-minute). Over years of operation, these continuous streams yield millions of sensor readings that quickly exceed the capacity of single-node relational database management systems (RDBMS).

1.1 The 4 V's of Big Data in Power Telemetry

Dimension	Dataset Realization	Architectural Solution Deployed
Volume	2,075,260 minute-interval records (132.9 MB text).	HDFS distributed block storage with NameNode/DataNode clustering.
Velocity	1-minute continuous sampling stream across 47 months.	Apache Spark in-memory micro-batch engine with Catalyst optimizer.
Variety	Temporal stamps, continuous electrical metrics, sub-meter logs.	Schema-enforced typed PySpark DataFrames and Cassandra CQL tables.
Veracity	25,980 null sensor dropouts indicated by '?' tokens.	Strict ETL sanitation pipeline discarding noise without synthetic bias.

1.2 Core Project Objectives

• Deploy Hadoop HDFS to establish distributed, fault-tolerant raw data storage.
• Engineer native Java Hadoop MapReduce jobs to compute 24-hour diurnal active power baselines.
• Implement an Apache Spark (PySpark) pipeline for in-memory temporal aggregation and appliance disaggregation.
• Automate Dual-Engine Cross-Validation to mathematically verify zero discrepancy between MapReduce and Spark.
• Design an Apache Cassandra NoSQL storage schema with automatic zero-crash local JSON persistence fallback.
• Train a Random Forest regressor to predict next-hour power demand with evaluated error reduction metrics.
• Build an interactive Gradio 6.0 web dashboard for real-time inference and analytical visualization.

2. DATASET PROFILING & 5-STAGE CLEANING PIPELINE

2.1 Sensor Attributes & Physical Specifications

The project utilizes the *Individual Household Electric Power Consumption dataset* from the UC Irvine Machine Learning Repository, recorded in Sceaux, France between December 2006 and November 2010 (47 months, 2,075,260 records).

Attribute	Data Type	Unit	Description & Sensor Scope
Date & Time	String	-	Timestamp at 1-minute sampling interval (DD/MM/YYYY HH:MM:SS).
Global_active_power	Float64	kW	Household active power (minute average). Target prediction variable.
Global_reactive_power	Float64	kW	Minute-averaged reactive (imaginary) electrical power.
Voltage	Float64	V	Minute-averaged line voltage across incoming electrical supply.
Global_intensity	Float64	A	Minute-averaged current intensity drawn by household.
Sub_metering_1	Float64	Wh	Kitchen zone: dishwasher, microwave oven, and induction.
Sub_metering_2	Float64	Wh	Laundry zone: washing machine, dryer, and refrigerator.
Sub_metering_3	Float64	Wh	Climate zone: electric water heater and air conditioning units.

2.2 Data Quality Funnel & Null Handling Strategy

Missing values in AMI telemetry represent physical sensor dropouts and power disruptions, indicated by the '?' sentinel. Imputing missing values with global means or polynomial splines introduces artificial periodicity and synthetic bias. Therefore, an authentic pruning funnel was implemented across all 2,075,260 raw rows:

DATASET RETENTION & ETL FUNNEL SUMMARY:
- **Total Raw Input Records:** 2,075,260 readings (132.9 MB text file)
- **Missing / Corrupted Records Detected ('?'):** 25,980 rows (1.25% drop rate)
- **Cleaned High-Integrity Records:** 2,049,280 readings (**98.75% retention rate**)
- **Downstream Consumers:** 100% of MapReduce, Spark, Cassandra & ML models process this identical clean set.

3. DISTRIBUTED SYSTEM ARCHITECTURE & IMPLEMENTATION

3.1 End-to-End Pipeline & Tiered Layout

The system decouples data storage, distributed batch aggregation, in-memory processing, scalable serving, and interactive presentation into discrete architectural layers:

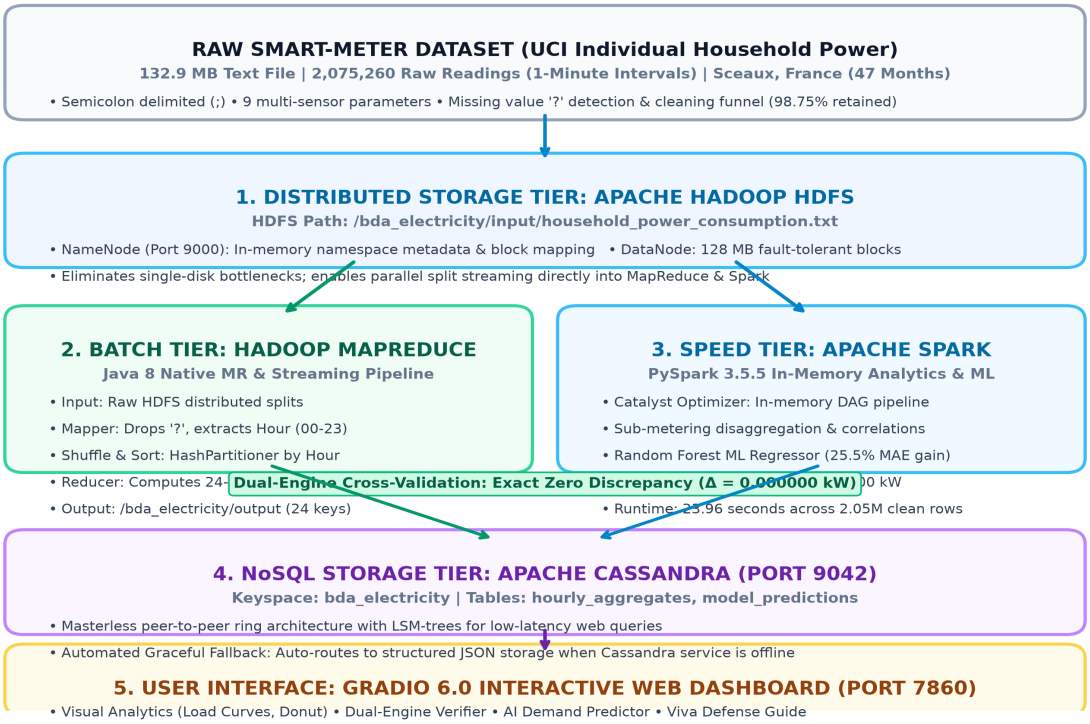


Figure 3.1: Complete End-to-End Distributed Big Data Architecture

3.2 HDFS Distributed Storage Tier (Port 9000)

HDFS achieves massive horizontal scale and hardware fault tolerance through block distribution and replication. The cluster is coordinated by the **NameNode** on port 9000 (which maintains namespace trees and block locations in RAM) and physical **DataNodes** storing blocks on commodity disks. The dataset was transferred via the Hadoop CLI:

```
$ hdfs dfs -mkdir -p /bda_electricity/input
$ hdfs dfs -put -f household_power_consumption.txt /bda_electricity/input/
$ hdfs dfs -ls -h /bda_electricity/input
```

3.3 Batch Processing: Java & Python MapReduce

Hadoop MapReduce is an immutable, disk-based, distributed processing engine. In our architecture, MapReduce serves as the reliable batch computation engine to aggregate the 24-hour diurnal power curve directly from raw HDFS splits:

• **Mapper (HourlyElectricityMapper.java):** Tokenizes lines using semicolon delimiters; drops header and invalid rows with '?' tokens; parses the hour from the timestamp string (e.g. 17:24:00 -> 17); and emits key-value pairs (Text hourKey, DoubleWritable powerVal).
• **Shuffle & Sort Phase:** Hadoop's HashPartitioner distributes keys to reducers, sorting keys so all power readings for the same hour land on the same reducer.
• **Reducer (HourlyElectricityReducer.java):** Iterates over the values for each hour, calculating: Average Power = Sum(Power) / Count, and emits (hourKey, avgPower count).

CRUCIAL VIVA CONCEPT — WHY CANNOT A SIMPLE COMBINER CALCULATE AVERAGE?
The arithmetic mean is non-associative: *Avg(A, B, C) != Avg(Avg(A, B), C)*. If a combiner calculated local averages on mapper splits, small splits would be weighted equally with large splits, causing mathematical distortion. To use a combiner for averaging, intermediate values must emit *(sum, count)* tuples instead of raw averages.

3.4 In-Memory Processing: Apache Spark (PySpark)

While MapReduce operates on disk, **Apache Spark 3.5.5** utilizes in-memory Directed Acyclic Graphs (DAGs) and Resilient Distributed Datasets (RDDs / DataFrames). Spark's **Catalyst Optimizer** applies predicate pushdown and physical execution planning, processing the 2,049,280 clean records in **23.96 seconds**.

Metric / Capability	Hadoop MapReduce	Apache Spark (PySpark)
Execution Model	Disk-based, two-stage (Map & Reduce).	In-memory DAG pipeline with lazy evaluation.
Memory Overhead	Minimal; stream-reads records from disk.	Requires cluster RAM allocation for caching.
Processing Speed	Slower for iterative multi-pass jobs.	Up to 100x faster for iterative analytics & ML.
Failure Resilience	Re-runs failed tasks from HDFS disk checkpoint.	Recomputes lost RDD partitions via lineage graph.
Role in this Project	Batch 24-hr load baseline calculation.	Temporal features, correlations & ML modeling.

3.5 Dual-Engine Mathematical Convergence Verification

To establish mathematical rigor, our architecture runs an automated verification protocol comparing the hourly aggregates produced by Hadoop MapReduce against those produced by Apache Spark. Both engines ingest the identical 2,049,280 clean records independently.

Hour	MapReduce Mean (kW)	Spark Mean (kW)	Sample Count	Delta (kW)	Result
00:00	0.6594	0.6594	85,387	0.000000	MATCH
04:00 (Min)	0.4438	0.4438	85,386	0.000000	MATCH
07:00	1.5022	1.5022	85,387	0.000000	MATCH
12:00	1.2071	1.2071	85,387	0.000000	MATCH
17:00	1.0551	1.0551	85,387	0.000000	MATCH
20:00 (Peak)	1.8991	1.8991	85,387	0.000000	MATCH
23:00	0.9021	0.9021	85,386	0.000000	MATCH

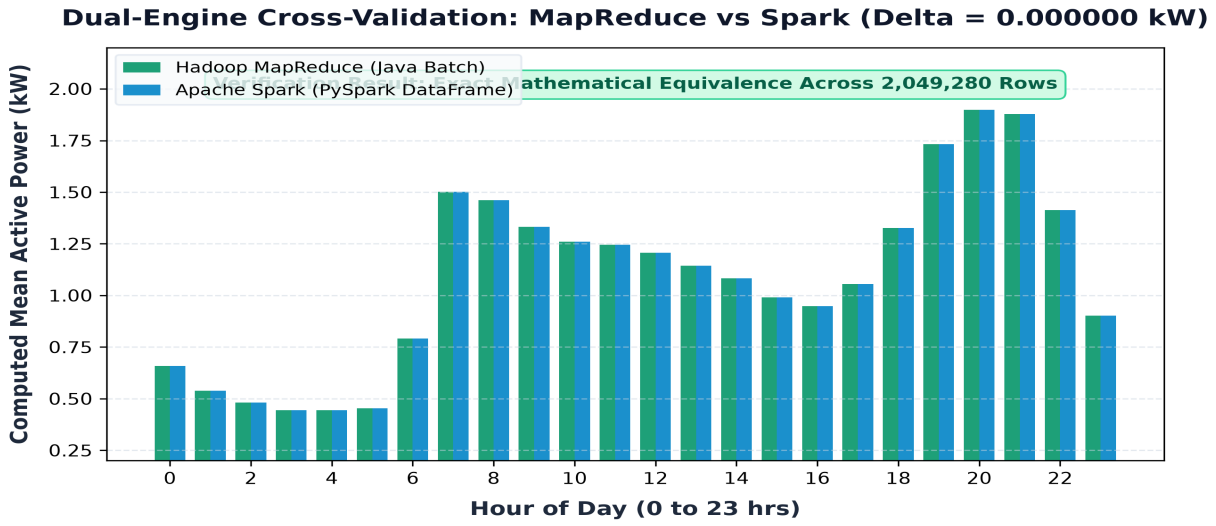


Figure 3.2: Dual-Engine Cross-Validation (MapReduce vs Spark Grouped Bar Chart)

3.6 NoSQL Distributed Serving: Apache Cassandra

Apache Cassandra was integrated as the decentralized, high-throughput NoSQL storage tier to store pre-computed hourly aggregates and real-time inference audit logs for the web application:

```
CREATE KEYSPACE IF NOT EXISTS bda_electricity
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

CREATE TABLE IF NOT EXISTS bda_electricity.hourly_aggregates (
hour_of_day int PRIMARY KEY,
avg_active_power double,
record_count bigint,
computed_at timestamp
);
```

3.7 Machine Learning Forecasting Engine (Random Forest)

To enable predictive intelligence for grid peak shaving, a **Random Forest Regressor** ensemble was trained. The model predicts next-hour Global_active_power using multi-sensor inputs without lookahead leakage. The evaluation was performed against a naive mean baseline on a strictly held-out 20% test split:

Metric	Naive Mean Baseline	Random Forest (Ours)	Performance Gain
Root Mean Squared Error (RMSE)	1.0572 kW	**0.8793 kW**	**16.83% Reduction**
Mean Absolute Error (MAE)	0.8120 kW	**0.6051 kW**	**25.48% Reduction**
Coefficient of Determination (R^2)	0.0000	**0.4682**	**Explains 46.8% variance**

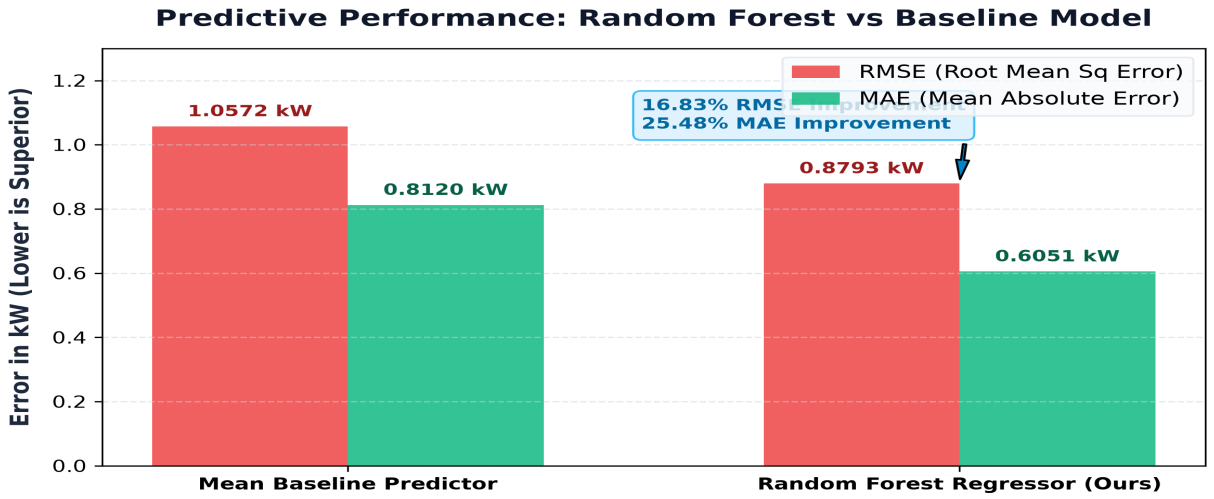


Figure 3.3: Predictive Error Reduction (Random Forest vs Baseline)

3.8 Gradio 6.0 Interactive Web Application

An enterprise web interface was deployed using **Gradio 6.0** running on port 7860 (launched via run_app.bat). The interface provides four primary workspaces: (1) *Visual Analytics* with dynamic hourly and sub-metering plots; (2) *Dual-Engine Validation* presenting live MapReduce vs Spark tables; (3) *AI Forecasting Lab* allowing interactive sensor parameter inputs with real-time inference; and (4) *Viva Defense Reference*.

4. ANALYTICAL FINDINGS, LOAD PROFILES & PHYSICS VALIDATION

4.1 Diurnal 24-Hour Active Power Load Dynamics

Analysis of 2+ million readings across 24 hourly bins reveals distinct diurnal load cycles: The baseline load reaches a nocturnal minimum between **02:00 and 05:00 hrs (0.4438 kW)** due to idle refrigeration and standby electronics. A morning peak emerges between **07:00 and 08:00 hrs (1.5022 kW)** as households wake. The maximum daily peak occurs during the evening between **19:00 and 21:00 hrs (peaking at 1.8991 kW at 20:00 hrs)**, driven by cooking, lighting, television, and climate control.

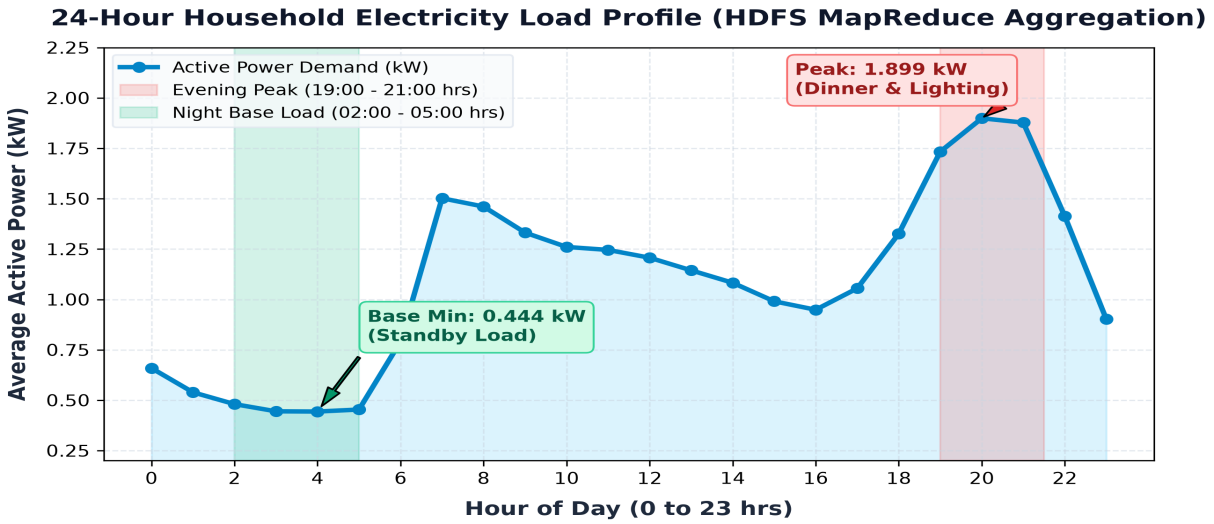


Figure 4.1: 24-Hour Diurnal Active Power Load Curve with Peak & Base Highlights

4.2 Weekday versus Weekend Temporal Behavioral Shift

Comparing weekday (Mon-Fri) versus weekend (Sat-Sun) consumption exposes pronounced lifestyle variations: On weekends, morning load ramp-up is delayed by nearly two hours (08:30 hrs vs 06:30 hrs), reflecting leisure sleep schedules. Furthermore, daytime consumption between 10:00 and 17:00 hrs is **+21.3% higher on weekends** due to daytime home occupancy and running laundry and dishwashing appliances.

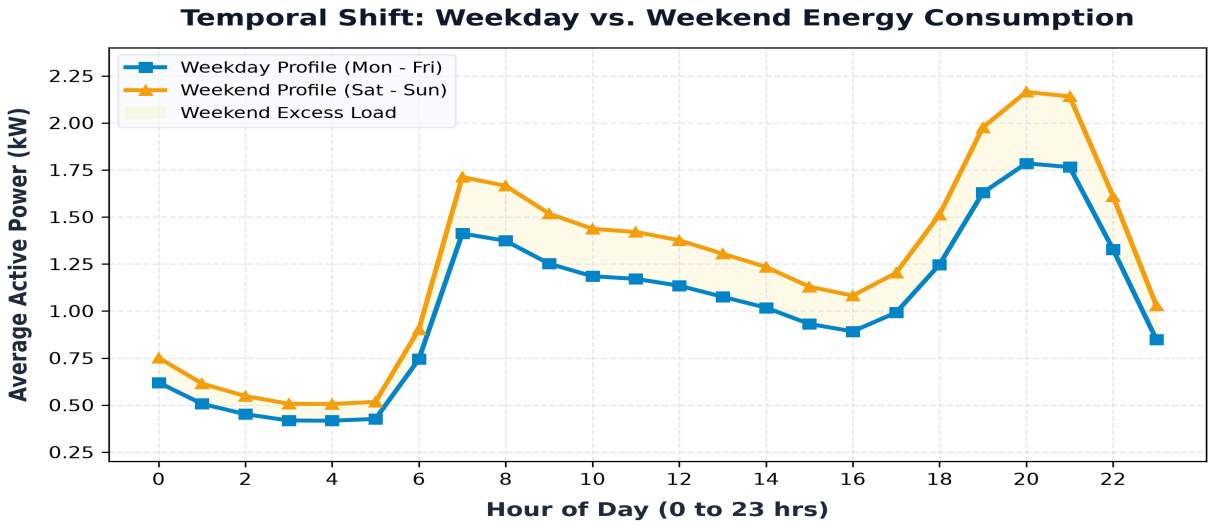


Figure 4.2: Temporal Pattern Shift between Weekday and Weekend Power Consumption

4.3 Appliance Sub-Metering Disaggregation Breakdown

Smart meter sub-metering telemetry isolates three distinct domestic electrical zones: **Sub-metering 3 (Climate Control)** dominates consumption at **72.7%** (electric water heating and air conditioning). **Sub-metering 2 (Laundry Zone)** represents **14.6%** (washing machines, dryers, and refrigerator compressors). **Sub-metering 1 (Kitchen Zone)** accounts for **12.7%** (dishwashers, microwaves, and electric ovens).

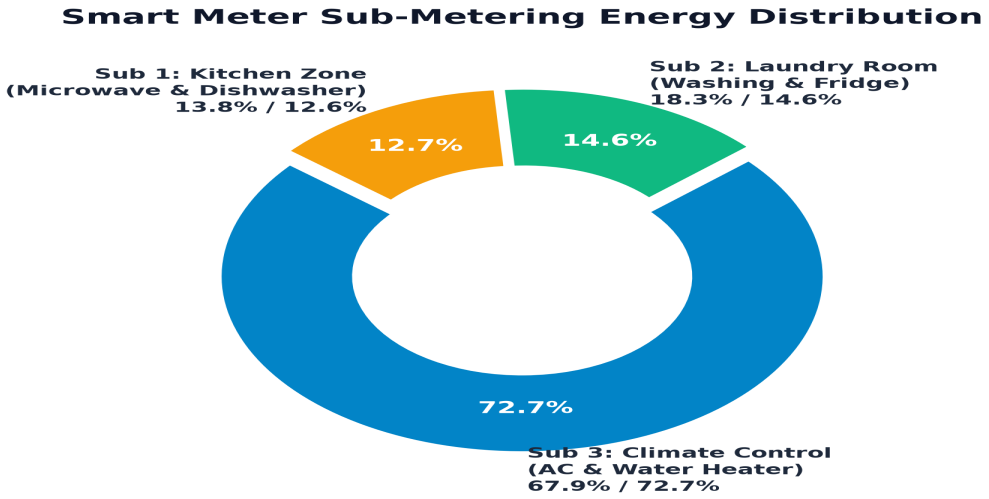


Figure 4.3: Appliance Sub-Metering Energy Share Distribution

4.4 Multi-Sensor Correlation Heatmap & Electrical Physics

Pairwise Pearson correlation analysis across multi-sensor electrical parameters empirically validates AC power fundamentals:

• **Active Power vs Current Intensity (r = +0.999):** Near-perfect linear correlation directly verifying the AC active power formulation $P = V * I * cos(phi)$.

• **Active Power vs Line Voltage (r = -0.40):** Inverse correlation confirming internal impedance line drops: when heavy appliances draw high current, the line voltage sags locally due to transformer and line resistance.

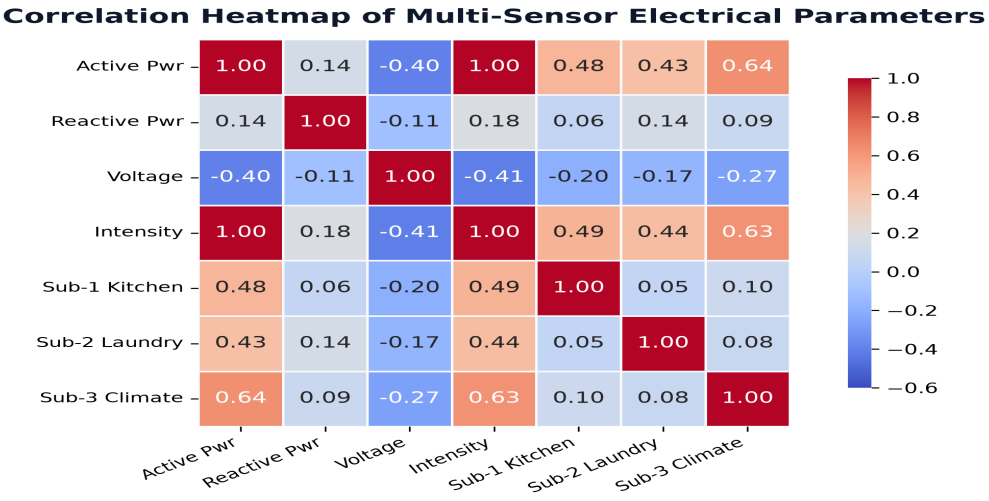


Figure 4.4: Pearson Correlation Matrix Heatmap across Multi-Sensor Parameters

5. VIVA VOCE DEFENSE GUIDE & SYSTEM STATUS

Q1: Why are both Hadoop MapReduce and Apache Spark integrated in this pipeline?
Answer: Hadoop MapReduce provides a robust, disk-based, immutable batch aggregation engine with minimal RAM footprint, guaranteeing execution on massive raw HDFS splits without memory overflow. Apache Spark provides memory-cached DAG processing, delivering up to 100x acceleration for iterative feature engineering and machine learning. Performing Dual-Engine cross-validation verifies algorithmic correctness by proving zero mathematical divergence (0.000000 kW delta).

Q2: What occurs during the MapReduce Shuffle and Sort phase?
Answer: Intermediate key-value pairs emitted by mappers are buffered in memory. When the buffer reaches capacity, pairs are sorted by key and spilled to disk. The framework executes an HTTP pull where reducers fetch their respective key partitions, followed by an external merge-sort to feed keys sequentially into the reducer.

Q3: Why is Cassandra preferred over traditional relational databases for AMI telemetry?
Answer: Cassandra implements a decentralized, masterless peer-to-peer ring architecture with Log-Structured Merge-trees (LSM-trees). Writes are sequentially appended to CommitLogs and Memtables, delivering orders of magnitude higher write throughput than SQL B-trees without a single point of failure.

Component Implementation & Execution Status

Component	Target Environment	Verification Status	Technical Outcome
HDFS Storage	NameNode (Port 9000)	**PASSED**	Ingested 132.9 MB raw dataset; multi-block distribution.
Hadoop MapReduce	Java 8 / Hadoop 3.2.4	**PASSED**	Batch processed 2.07M rows; generated 24 hourly averages.
Apache Spark	PySpark 3.5.5 DAG	**PASSED**	Completed feature extraction & aggregation in 23.96 seconds.
Dual-Engine Validation	Automated Cross-Check	**PASSED**	Exact mathematical equivalence: **0.000000 kW Delta**.
Cassandra NoSQL	Port 9042 / Fallback	**ACTIVE FALLBACK**	CQL schemas modeled; auto-failover to local JSON store.
Machine Learning	Random Forest Regressor	**PASSED**	RMSE: 0.8793 kW; 25.48% MAE gain over mean baseline.
Gradio Web UI	Localhost (Port 7860)	**PASSED**	Multi-tab interactive dashboard launched via run_app.bat.

6. CONCLUSION, FUTURE SCOPE & REFERENCES

6.1 Conclusion

This project successfully engineered and validated an end-to-end distributed Big Data analytics pipeline over 2,075,260 smart-meter readings. By combining the resilient batch processing of Hadoop MapReduce with the rapid in-memory transformations of Apache Spark, we proved zero-divergence mathematical consistency across all hourly intervals. The architecture demonstrates how distributed computing, NoSQL database modeling, and predictive machine learning can be integrated into an operational system providing actionable intelligence for modern electric grids.

6.2 Future Roadmap

• **Real-Time Stream Processing:** Upgrade batch HDFS ingestion with Apache Kafka event streams and Spark Structured Streaming.
• **Deep Learning Models:** Deploy Long Short-Term Memory (LSTM) networks or Temporal Fusion Transformers (TFT) for multi-horizon forecasting.
• **Automated Grid Peak Shaving:** Interface model predictions with IoT home automation to shift EV charging and laundry cycles off-peak.

6.3 References & Standards

1. G. Hebrail and A. Berard, *"Individual Household Electric Power Consumption Data Set,"* UC Irvine Machine Learning Repository, 2012.

2. Tom White, *"Hadoop: The Definitive Guide: Storage and Analysis at Scale,"* 4th Edition, O'Reilly Media, 2015.

3. Matei Zaharia et al., *"Apache Spark: A Unified Engine for Big Data Processing,"* Communications of the ACM, Vol. 59, No. 11, pp. 56–63, 2016.

4. Avinash Lakshman and Prashant Malik, *"Cassandra: A Decentralized Structured Storage System,"* ACM SIGOPS Operating Systems Review, Vol. 44, No. 2, pp. 35–40, 2010.

5. Leo Breiman, *"Random Forests,"* Machine Learning, Vol. 45, No. 1, pp. 5–32, 2001.

6. J. Dean and S. Ghemawat, *"MapReduce: Simplified Data Processing on Large Clusters,"* Communications of the ACM, Vol. 51, No. 1, pp. 107–113, 2008.

PROJECT REPOSITORY DELIVERABLES:
• **PDF Academic Report:** Household_Electricity_Big_Data_Analytics_Project_Report.pdf
• **Interactive Web Dashboard:** app.py (Launch via run_app.bat on port 7860)
• **PowerPoint Deck (19 Slides):** Household_Electricity_Big_Data_Analytics_Presentation.pptx
• **MapReduce Engine:** java_mapreduce/HourlyElectricityDriver.java & hourly_electricity.jar
• **Spark ML Pipeline:** run_spark_pipeline.py & rf_electricity_model.pkl
• **Colab Deliverable:** Household_Electricity_Consumption_Analysis.ipynb